

Player object text components in server status messages (MotD) are no longer replaced by fallback text

[MC-306860](#) NETWORKING

Status	Resolved / Fixed
Affected	26.1 Pre-Release 2
Fixed in	26.1 Pre-Release 3
Reported	2026-03-14
Mojira category	Networking

REPRODUCTION TARGET

Use version:

26.1 Pre-Release 2

Affected versions (1):

26.1 Pre-Release 2

Environment:

Minecraft Java Edition 26.1 Pre-Release 2 client and a Python 3 mock Minecraft status server running on the same computer. The mock server listens on TCP port 25565 and is accessed through the loopback address 127.0.0.1. A vanilla or dedicated Minecraft server is not required, and the client must not join the mock server; the reproduction occurs in the Multiplayer server list.

PRECONDITIONS

- Minecraft Java Edition 26.1 Pre-Release 2 is installed and selectable in the Minecraft Launcher.
- Python 3 is installed, and the ``python`` command invokes it from Command Prompt or PowerShell.
- TCP port 25565 is available; stop any Minecraft server or other program already using that port before starting the mock server.
- The Minecraft client and Python script will run on the same computer so that the exact address 127.0.0.1 reaches the mock server.
- The executor can create and save a plain-text `.py`` file.
- If the operating system displays a firewall prompt for Python, permit the local connection needed for the test.
- Keep the terminal running after starting the script. Closing it stops the mock server.

STEPS TO REPRODUCE

1 INFERRED

Open Command Prompt or PowerShell and verify that the Python command is available.

INPUT `python --version`

SUCCESS CUE The terminal prints a Python 3 version instead of reporting that `python` is unknown or unavailable.

2 INFERRED

Open a plain-text editor and create a new file named `mc306860\_mock\_status.py`. Ensure the file is saved with the `.py` extension rather than `.py.txt`.

INPUT `mc306860_mock_status.py`

SUCCESS CUE A writable plain-text file named `mc306860\_mock\_status.py` exists.

3 REPORT

Paste the report's mock status-server script into `mc306860\_mock\_status.py` exactly as supplied, then save the file.

INPUT

```
import socket import json def encode_varint(value): result = bytearray() while True: byte = value & 0x7F value >>= 7 if value != 0: byte |= 0x80 result.append(byte) if value == 0: break return bytes(result) def decode_varint(data, pos=0): result = 0 shift = 0 while pos < len(data): byte = data[pos] pos += 1 result |= (byte & 0x7F) << shift shift += 7 if not (byte & 0x80): return result, pos raise ValueError("Ran out of data reading VarInt") def encode_string(text): encoded = text.encode('utf-8') return encode_varint(len(encoded)) + encoded def build_packet(packet_id, data=b''): id_bytes = encode_varint(packet_id) content = id_bytes + data return encode_varint(len(content)) + content def recv_exact(sock, n): data = b'' while len(data) < n: chunk = sock.recv(n - len(data)) if not chunk: raise ConnectionError("Client disconnected early") data += chunk return data def recv_packet(sock): length_bytes = bytearray() while True: byte = recv_exact(sock, 1)[0] length_bytes.append(byte) if not (byte & 0x80): break if len(length_bytes) >= 5: raise ValueError("VarInt too long") length, _ = decode_varint(bytes(length_bytes)) content = recv_exact(sock, length) packet_id, offset = decode_varint(content) payload = content[offset:] return packet_id, payload STATUS_JSON = { "description": [ {"player": {"name": "alex"}}, {"sprite": "block/stone"} ], "players": {"max": 20, "online": 0}, "version": {"name": "26.1 Pre-Release 2", "protocol": 1073742123}, "enforcesSecureChat": True } def handle_client(conn, addr): print(f"\n[+] Client connected from {addr}") try: packet_id, payload = recv_packet(conn) if packet_id != 0x00: return pos = 0 protocol_ver, pos = decode_varint(payload, pos) addr_len, pos = decode_varint(payload, pos) server_addr = payload[pos:pos + addr_len].decode('utf-8') pos += addr_len server_port = int.from_bytes(payload[pos:pos+2], 'big') pos += 2 next_state, pos = decode_varint(payload, pos) if next_state != 1: return packet_id, _ = recv_packet(conn) json_str = json.dumps(STATUS_JSON) response_packet = build_packet(0x00, encode_string(json_str)) conn.sendall(response_packet) print(f" Payload sent: {json_str}") try: packet_id, ping_payload = recv_packet(conn) if packet_id == 0x01: conn.sendall(build_packet(0x01, ping_payload)) except Exception: pass except Exception as e: print(f" Error: {e}") finally: conn.close() print(f"[-] Client disconnected.") def main(): server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) server.bind(('0.0.0.0', 25565)) server.listen(5) print("Mock server running on port 25565. Add 127.0.0.1 in Minecraft.") print("Press Ctrl+C to stop.") try: while True: conn, addr = server.accept() handle_client(conn, addr) except KeyboardInterrupt: print("\nStopped.") finally: server.close() if __name__ == '__main__': main()
```

SUCCESS CUE The file is saved, begins with `import socket`, and ends with the call to `main()`.

4 INFERRED

Open Command Prompt or PowerShell in the directory containing `mc306860\_mock\_status.py`. One way to do this on Windows is to open that folder in File Explorer, click the address bar, type `powershell`, and press Enter.

INPUT powershell

SUCCESS CUE The terminal's current directory is the folder containing `mc306860\_mock\_status.py`.

5 REPORT

Start the mock status server by executing the saved Python file. Do not close this terminal after the command starts.

INPUT python mc306860_mock_status.py

SUCCESS CUE The terminal displays `Mock server running on port 25565. Add 127.0.0.1 in Minecraft.` followed by `Press Ctrl+C to stop.` and does not return to the command prompt.

6 INFERRED

If the operating system displays a firewall prompt for Python, allow the connection. Leave the mock-server terminal open and running in the background.

SUCCESS CUE The Python process remains active without a traceback or an error stating that port 25565 could not be bound.

7 REPORT

Open the Minecraft Launcher, select the installation for Minecraft Java Edition `26.1 Pre-Release 2`, and launch it. Do not substitute the fixed version `26.1 Pre-Release 3`.

INPUT 26.1 Pre-Release 2

SUCCESS CUE The Minecraft Java Edition main menu is open and identifies the running version as 26.1 Pre-Release 2.

8 INFERRED

From the Minecraft main menu, click `Multiplayer`. If a standard pre-release or multiplayer warning screen appears, acknowledge it to continue to the Multiplayer server list.

SUCCESS CUE The Multiplayer server-list screen is visible.

9 REPORT

Click `Add Server` at the bottom of the Multiplayer screen.

SUCCESS CUE The Edit Server Info screen is open and contains fields for `Server Name` and `Server Address`.

10 INFERRED

In the `Server Name` field, enter a recognizable label for the mock server. This label is only for identifying the entry and is not part of the network payload.

INPUT MC-306860 Mock Server

SUCCESS CUE The Server Name field contains `MC-306860 Mock Server`.

11 **REPORT**

Click the `Server Address` field, remove any existing contents, and enter the loopback address exactly as supplied in the report. Do not add `http://`, a path, or spaces. Because the script uses the default Minecraft port 25565, no explicit port suffix is required.

INPUT `127.0.0.1`

SUCCESS CUE The Server Address field contains exactly `127.0.0.1`.

12 **REPORT** **TRIGGER**

Click `Done` once to save the server entry and return to the Multiplayer server list. This causes the client to request status information from the Python mock server.

SUCCESS CUE The Multiplayer list is visible and contains the saved entry named `MC-306860 Mock Server`.

13 **REPORT**

Do not double-click the server or press `Join Server`. Wait for the server-list status request to complete. Use the terminal as the readiness signal: wait until it prints a client connection line and a `Payload sent:` line for the status JSON.

SUCCESS CUE The Python terminal has printed `[+] Client connected from ...`, followed by `Payload sent: ...`, and the saved server entry has finished its status poll in the Multiplayer list.

14 **INFERRED**

Keep the Multiplayer list open with the completed mock-server entry visible. Do not refresh the list, join the server, stop the Python process, or navigate away before handing control to the observation subsystem.

SUCCESS CUE The mock server's status description has just been received and rendered beneath the saved server name, and the entry remains visible for inspection.

HANDOFF TO OBSERVATION

The client has returned to the Multiplayer server list after `Done` was clicked, the Python terminal has logged that it sent the supplied status payload, and the resulting status description is currently rendered beneath the `MC-306860 Mock Server` entry. Begin observation without joining the server or refreshing the list.

UNCERTAINTIES

- The report specifies that Python must be installed and uses the command form ``python _.py``, but it does not specify an operating system or Python installation method. This plan uses Windows-oriented Command Prompt or PowerShell instructions because those are explicitly named in the report.
- Some Windows Python installations expose Python 3 through ``py`` rather than ``python``. The report specifically instructs use of ``python``, so that command is retained; if it is unavailable, the environment must be corrected or the equivalent launcher selected before continuing.
- The report does not specify a server-list display name. ``MC-306860 Mock Server`` is an arbitrary practical label and does not affect the status payload.
- The report does not explicitly discuss firewall prompts or port conflicts. Allowing local Python networking and keeping TCP port 25565 free are practical requirements for executing the supplied script.
- No vanilla Minecraft server is used. The supplied Python program only implements enough of the status protocol to answer the Multiplayer server-list query.

MANUAL RESULT

Version used	
Reproduced?	YES / NO / PARTIAL
Trigger reached?	YES / NO
Crash file	
Notes	
Tested by / date	